

Cumplimiento OWASP Top 10 — CiberDojo

📅 21 sept 2026 · @Raul Chavez

La referencia usada es **OWASP Top 10:2021**, la última edición publicada y verificable de OWASP para aplicaciones web. No existe una edición "2026" publicada por OWASP al momento de este informe; si el lector tiene una fuente distinta, se debe ajustar la comparación.

Alcance y método

Se revisó el código fuente completo del repositorio `shield-ecuador-app` en su estado del commit `41569fc` (2026-09-21): la app de usuario en React/TypeScript (`frontend/`), la consola de administración en JavaScript plano (`central-admin-app/`), las funciones Edge de Supabase (`supabase/functions/`) y las 68 migraciones de base de datos (`supabase/migrations/`).

Método: lectura directa del código de cada punto de entrada (autenticación, subida de archivos, llamadas a servicios externos, construcción de HTML), `npm audit` sobre las dependencias de ambas apps, búsqueda de patrones de riesgo (`innerHTML` , `eval` , manejadores inline, URLs sin validar) y verificación de que cada tabla de la base de datos tiene Row Level Security (RLS) activado. No se hizo pentesting dinámico (sin intentos de explotación contra el sistema en producción); es una revisión de código y configuración.

Las 10 categorías (OWASP Top 10:2021)

#	Categoría	Estado	Evidencia
A01	Control de acceso roto	Cumple	Las 52 tablas de la base de datos tienen Row Level Security activado (verificado: 52 de 52). Rol <code>admin</code>

			protegido por trigger <pre>prevent_user_security_field_tampering (005_security_hardening.sql , 026_learning_progress.sql): un usuario no puede auto-asignarse el rol.</pre>
A02	Fallos criptográficos	Cumple	Autenticación de usuarios delegada a Supabase Auth (no hay hash de contraseñas propio). Comparación de credenciales de la consola admin con <code>crypto.timingSafeEqual (central-admin-app/server.js:202-204)</code>, evita ataques de temporización. Claves de terceros (proveedores de IA) guardadas en Supabase Vault, nunca devueltas al cliente.
A03	Inyección	Cumple	Sin SQL directo con texto del usuario (PostgREST parametrizado). HTML generado con <code>esc()</code> en ambas apps; búsqueda de <code>innerHTML</code> sin escapar y de <code>eval / new Function / document.write</code> en <code>frontend/src</code> no encontró casos. CSV exportado con <code>csvSafe()</code> para evitar inyección de fórmulas.
A04	Diseño inseguro	Cumple parcial	El sorteo del campeonato usa reclamo atómico de estado (<code>championship-draw-round1</code>) y las respuestas correctas no se exponen al cliente en <code>learning_state</code> . No hay modelado de amenazas documentado formalmente ni un registro de decisiones de diseño de seguridad fuera del código.
A05	Configuración de seguridad incorrecta	Cumple	Cabeceras de seguridad en ambos servidores: <code>Content-Security-Policy</code> (sin <code>unsafe-inline</code> en scripts), <code>Strict-Transport-Security</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> / <code>frontend/static</code>

			<code>POLICY , PERMISSIONS-POLICY (frontend/static-server.js:72-76 , central-admin-app/server.js:40-45)</code> . Scripts de terceros (echarts, echarts-gl) cargados con integridad de subrecurso (SRI).
A06	Componentes vulnerables y desactualizados	Cumple parcial	<code>npm audit --omit=dev</code> en frontend/: 0 vulnerabilidades en dependencias de producción. <code>npm audit</code> completo (incluye herramientas de desarrollo, p. ej. <code>react-scripts /webpack/jest</code>): 29 avisos (14 altas, 6 moderadas, 9 bajas), todos en el toolchain de build, no en el código que se sirve al navegador. <code>central-admin-app</code> no usa dependencias de npm en producción (servidor sin librerías externas).
A07	Fallos de identificación y autenticación	Cumple	Sesiones de usuario gestionadas por Supabase Auth. Consola admin: bloqueo tras 10 intentos fallidos por 10 minutos, contado solo cuando la petición trae credenciales (<code>central-admin-app/server.js:21,107</code>), evitando bloquear a quien nunca envió clave.
A08	Fallos de integridad de software y datos	Cumple	Scripts de CDN (echarts 5.5.1, echarts-gl 2.0.9) fijados a una versión exacta con hash SRI y <code>crossorigin="anonymous"</code> (<code>central-admin-app/index.html:11-12</code>): si el archivo remoto cambiara, el navegador lo rechaza.
A09	Fallos de registro y monitoreo	Cumple parcial	Existe un Centro de Seguridad con alertas, métricas y auditoría de eventos T-Pot (migraciones <code>052</code> , <code>054</code> , <code>055</code>). No se revisó retención de logs ni alertado ante intentos de fuerza bruta reales en producción (fuera del alcance de esta revisión de código).

A10	Server-Side Request Forgery (SSRF)	Cumple	Módulo propio <code>_shared/url-guard.ts</code> reutilizado en fuentes de noticias, URLs de proveedores de IA y webhooks de alerta: bloquea IPs privadas, exige HTTPS público, sigue redirecciones manualmente y limita el tamaño de respuesta (512 KB). Refuerzo del lado cliente con <code>safeHttpUrl / publicUrlProblem</code> para imágenes y enlaces de campañas de propaganda.
-----	------------------------------------	--------	---

Resumen: 7 de 10 categorías cumplen completamente, 3 cumplen parcialmente.
Ninguna categoría quedó sin cubrir.

Lo que no cumple o queda parcial

- 1. **A06 — herramientas de desarrollo desactualizadas.** El toolchain de build (`react-scripts` , `jest`, `webpack-dev-server`) trae 29 avisos de `npm audit` , 14 de severidad alta. No viajan al navegador del usuario final, pero un equipo que trabaje sobre este código los hereda. Riesgo bajo en producción, riesgo medio en el entorno de desarrollo (p. ej. `webpack-dev-server` expuesto en una red compartida).
- 2. **A09 — monitoreo sin verificar en producción real.** El Centro de Seguridad existe en el código, pero esta revisión no confirmó que las alertas realmente lleguen (webhook configurado, correo de destino válido) ni cuánto tiempo se conservan los registros.
- 3. **A04 — sin modelado de amenazas documentado.** Las decisiones de diseño de seguridad viven repartidas en migraciones y comentarios de código, no en un documento único que un nuevo integrante del equipo pueda leer de una vez.
- 4. **Fuera de las 10 categorías, pero relevante:** el reporte de la consola admin ("Reportes") limitaba el detalle a 5.000 filas por consulta; se corrigió para que los totales sean exactos, pero el detalle sigue con ese límite. No es una falla de seguridad, es una limitación de exactitud de datos en volúmenes muy altos.

Recomendaciones

Prioridad	Recomendación	Por qué
Alta	Actualizar o aislar el toolchain de build (<code>react-scripts</code> y dependencias)	Elimina los 14 avisos altos de <code>npm audit</code> ; hoy no afectan al usuario final pero sí a quién desarrolla
Alta	Confirmar en producción que las alertas del Centro de Seguridad llegan de verdad (webhook + correo)	El código está listo; falta la prueba end-to-end contra el sistema real
Media	Escribir un documento breve de modelado de amenazas (quién puede hacer qué, qué pasa si falla cada pieza externa)	Hoy esas decisiones solo viven en el código; un documento acelera la incorporación de nuevas personas al equipo
Media	Definir política de retención de logs de seguridad	No se encontró un límite documentado de cuánto tiempo se guardan los eventos de auditoría
Baja	Agregar una función SQL agregada para el reporte de Reportes, sin límite de 5.000 filas	Solo afecta la exactitud del detalle en períodos con mucho tráfico; los totales ya son exactos
Baja	Repetir este ejercicio contra la próxima edición formal de OWASP cuando se publique	La comparación aquí es contra OWASP Top 10:2021; conviene rehacerla si aparece una edición posterior verificable

Conclusión: el código cumple, con evidencia verificable, 7 de las 10 categorías del OWASP Top 10:2021 y cumple parcialmente las 3 restantes; ninguna quedó sin cubrir. Las brechas encontradas son de proceso (documentación, verificación en vivo) y de higiene de dependencias de desarrollo, no de diseño de seguridad del producto.

